

ACTIVIDAD DE APRENDIZAJE 4

Rey David Borrego Pérez

7502410022

Desarrollo de Software Web

Ingeniería de Software

Universidad de Cartagena

2026-1



Resumen

El presente informe describe el diseño, desarrollo e integración de una aplicación web frontend que funciona como cliente RESTful para consumir una API backend construida previamente con Spring Boot y arquitectura hexagonal. El proyecto fue desarrollado con React y Vite, utilizando Axios para realizar peticiones HTTP y React Router para organizar la navegación entre las vistas principales. La aplicación permite gestionar usuarios y pedidos mediante operaciones CRUD completas, incluyendo consulta, creación, actualización y eliminación de registros. Además, se implementaron validaciones básicas en formularios, manejo de mensajes de éxito y error, organización del código en componentes, páginas, modelos y servicios, y documentación del proceso mediante Git y GitHub. El resultado final evidencia la integración entre frontend y backend, cumpliendo los requerimientos de consumo de endpoints RESTful mediante los métodos GET, POST, PUT y DELETE.

1. Introducción

El desarrollo de aplicaciones web modernas requiere la integración entre interfaces gráficas de usuario y servicios backend expuestos mediante API RESTful. Esta separación permite que el frontend se encargue de la experiencia del usuario, mientras que el backend administra la lógica de negocio, la persistencia y las reglas del sistema.

En el marco de la asignatura Desarrollo de Software Web, correspondiente al programa de Ingeniería de Software de la Universidad de Cartagena, se desarrolló una aplicación frontend que consume los endpoints del backend construido en la Unidad 3. El objetivo principal fue implementar un cliente RESTful capaz de demostrar desde una interfaz gráfica las operaciones de creación, consulta, actualización y eliminación de registros.

El proyecto fue construido con React y Vite. React permitió estructurar la interfaz en componentes reutilizables, mientras que Vite facilitó la configuración del entorno de desarrollo. Para el consumo de la API se empleó Axios, centralizando las peticiones HTTP en servicios independientes.

El repositorio del proyecto frontend es: <https://github.com/ReyDavid07/unidad4-react-rest-client.git>

2. Marco Teórico

2.1. Cliente RESTful Web

Un cliente RESTful web es una aplicación frontend que consume servicios expuestos por un backend a través del protocolo HTTP. En este tipo de arquitectura, el cliente realiza solicitudes utilizando métodos como GET, POST, PUT y DELETE para consultar o modificar recursos del sistema. Este enfoque permite desacoplar la interfaz de usuario de la lógica del servidor y facilita la reutilización del backend por distintos tipos de clientes.

2.2. API REST

REST, Representational State Transfer, es un estilo arquitectónico descrito por Fielding (2000) para el diseño de sistemas distribuidos. En el contexto de aplicaciones web, una API REST organiza la comunicación alrededor de recursos identificados por URLs y manipulados mediante métodos HTTP. La adopción de este estilo facilita la interoperabilidad entre aplicaciones y la construcción de sistemas escalables.

2.3. React

React es una biblioteca de JavaScript orientada a la construcción de interfaces de usuario. Su modelo basado en componentes permite dividir la aplicación en piezas reutilizables que administran su propio estado y comportamiento. En este proyecto, React se utilizó para construir las vistas de usuarios, pedidos, inicio y navegación principal.

2.4. Axios

Axios es una biblioteca utilizada para realizar solicitudes HTTP desde aplicaciones JavaScript. En el proyecto se configuró una instancia centralizada de Axios con la URL base del backend, encabezados JSON e interceptores de respuesta para manejar errores de forma uniforme.

2.5. Control de versiones con Git y GitHub

Git permitió gestionar el historial de cambios del proyecto, mientras que GitHub se utilizó para alojar el repositorio remoto y documentar el proceso de desarrollo mediante ramas y

Pull Requests. Este flujo de trabajo permitió organizar cada avance por funcionalidad y mantener una trazabilidad clara del proyecto.

3. Objetivos

3.1. Objetivo General

Desarrollar una aplicación web tipo cliente RESTful que consuma los endpoints de la API backend de la Unidad 3 y permita realizar operaciones CRUD desde una interfaz gráfica desarrollada con React.

3.2. Objetivos Específicos

- Implementar una aplicación frontend utilizando React y Vite.
- Configurar un cliente HTTP para consumir la API RESTful del backend.
- Consumir endpoints mediante los métodos GET, POST, PUT y DELETE.
- Implementar operaciones CRUD para usuarios y pedidos.
- Validar formularios antes del envío de información al backend.
- Mostrar mensajes de éxito y error de acuerdo con las respuestas del servidor.
- Organizar el proyecto en componentes, páginas, servicios, modelos y configuración.
- Documentar la integración entre el frontend y el backend mediante informe y repositorio GitHub.

4. Tecnologías Utilizadas

4.1. React

React fue seleccionado como tecnología principal para el frontend debido a su enfoque modular, su amplio uso en el desarrollo web actual y su facilidad para construir interfaces dinámicas basadas en componentes.

4.2. Vite

Vite fue utilizado como herramienta de creación y ejecución del proyecto frontend. Su configuración ligera permitió iniciar rápidamente el entorno de desarrollo y trabajar con recarga en caliente durante la implementación.

4.3. Axios

Axios fue utilizado para enviar solicitudes HTTP hacia el backend. La aplicación centraliza su configuración en el archivo `axiosConfig.js`, lo que facilita mantener una URL base única y manejar errores de forma consistente.

4.4. React Router DOM

React Router DOM permitió configurar la navegación entre las páginas principales del cliente RESTful, tales como inicio, login, usuarios y pedidos.

4.5. Git y GitHub

Git y GitHub se utilizaron para controlar versiones, publicar el código fuente y evidenciar el proceso mediante commits, ramas y Pull Requests.

5. Desarrollo del Sistema

5.1. Descripción general del backend consumido

El backend consumido corresponde al proyecto de la Unidad 3, desarrollado con Spring Boot y arquitectura hexagonal. Este backend expone endpoints RESTful para la gestión de usuarios y pedidos, los cuales son consumidos desde el frontend mediante solicitudes HTTP.

La URL base utilizada para el consumo del API fue `http://localhost:8080/api`. Esta configuración se almacenó en el archivo `.env` mediante la variable `VITE_API_BASE_URL`, permitiendo separar la configuración del código fuente.

```
VITE_API_BASE_URL=http://localhost:8080/api
```

Figura 1. Configuración de la URL base del API en el archivo .env.

5.2. Estructura general del proyecto frontend

La aplicación fue organizada siguiendo una separación por responsabilidades. Las páginas contienen las vistas principales, los servicios encapsulan el consumo de endpoints, los modelos agrupan estructuras iniciales y validaciones, los componentes contienen elementos reutilizables y la carpeta api centraliza la configuración de Axios.

```
src/  
├── api/  
│   └── axiosConfig.js  
├── components/  
│   └── Navbar.jsx  
├── models/  
│   ├── pedidoModel.js  
│   └── userModel.js  
├── pages/  
│   ├── Home.jsx  
│   ├── Login.jsx  
│   ├── PedidosPage.jsx  
│   └── UsersPage.jsx  
├── services/  
│   ├── authService.js  
│   ├── pedidoService.js  
│   └── userService.js  
├── App.jsx  
├── main.jsx  
└── styles.css
```

Figura 2. Estructura del código fuente del cliente RESTful.

5.3. Tabla de endpoints consumidos

Módulo	Método	Endpoint	Descripción	Datos enviados
Usuarios	GET	/api/users	Listar usuarios	No aplica
Usuarios	GET	/api/users/{id}	Consultar usuario por ID	ID por URL
Usuarios	POST	/api/users	Crear usuario	name, email, password, role
Usuarios	PUT	/api/users/{id}	Actualizar usuario	ID y datos actualizados
Usuarios	DELETE	/api/users/{id}	Eliminar usuario	ID por URL
Pedidos	GET	/api/pedidos	Listar pedidos	No aplica
Pedidos	GET	/api/pedidos/{id}	Consultar pedido por ID	ID por URL
Pedidos	POST	/api/pedidos	Crear pedido	numeroPedido, cliente, producto, cantidad, valorUnitario, fecha
Pedidos	PUT	/api/pedidos/{id}	Actualizar pedido	ID y datos actualizados
Pedidos	DELETE	/api/pedidos/{id}	Eliminar pedido	ID por URL

5.4. Configuración del cliente HTTP

La configuración del cliente HTTP se realizó en el archivo axiosConfig.js. Allí se definió la URL base de la API, el encabezado Content-Type para trabajar con JSON, un tiempo máximo de espera y un interceptor para transformar errores del servidor en mensajes legibles para el usuario.

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:8080/api',
  headers: { 'Content-Type': 'application/json' },
  timeout: 10000
})
```

```
}}
```

Figura 3. Fragmento de configuración de Axios para el consumo RESTful.

5.5. Servicios REST implementados

Los servicios REST se implementaron en archivos independientes para evitar que las páginas dependan directamente de los detalles de las peticiones HTTP. Esta separación facilita el mantenimiento y permite modificar la URL, los endpoints o la lógica de consumo sin afectar la estructura visual de la aplicación.

```
export const userService = {
  async findAll() { return (await api.get('/users')).data },
  async findById(id) { return (await api.get(`/users/${id}`)).data },
  async create(payload) { return (await api.post('/users', payload)).data },
  async update(id, payload) { return (await api.put(`/users/${id}`, payload)).data },
  async remove(id) { await api.delete(`/users/${id}`) }
}
```

Figura 4. Servicio de usuarios con operaciones CRUD.

```
export const pedidoService = {
  async findAll() { return (await api.get('/pedidos')).data },
  async findById(id) { return (await api.get(`/pedidos/${id}`)).data },
  async create(payload) { return (await api.post('/pedidos', payload)).data },
  async update(id, payload) { return (await api.put(`/pedidos/${id}`, payload)).data },
  async remove(id) { await api.delete(`/pedidos/${id}`) }
}
```

Figura 5. Servicio de pedidos con operaciones CRUD.

5.6. Interfaz gráfica y validaciones

La interfaz gráfica se organizó en páginas para usuarios y pedidos. Cada página contiene un formulario para crear o editar registros, una tabla para listar los datos y botones de acción para actualizar o eliminar registros. Antes del envío de información, los formularios realizan validaciones básicas para evitar peticiones incompletas.

Además, la aplicación muestra mensajes de éxito cuando una operación se completa correctamente y mensajes de error cuando el backend responde con fallos o cuando la comunicación no puede establecerse.

6. Pruebas Realizadas

La verificación del funcionamiento se planteó mediante pruebas funcionales manuales, orientadas a validar la comunicación entre el cliente React y el backend RESTful. Estas pruebas cubren los métodos HTTP principales y la respuesta visual de la aplicación frente a operaciones exitosas y fallidas.

- Ejecución del backend en el puerto 8080 y validación de la URL base del API.
- Ejecución del frontend en el puerto 5173 mediante npm run dev.
- Prueba del listado de usuarios mediante GET /api/users.
- Prueba de creación de usuarios mediante POST /api/users.
- Prueba de actualización de usuarios mediante PUT /api/users/{id}.
- Prueba de eliminación de usuarios mediante DELETE /api/users/{id}.
- Prueba del CRUD completo de pedidos mediante los endpoints /api/pedidos.
- Verificación de mensajes de éxito y error en la interfaz.
- Verificación de validaciones básicas en formularios antes del envío.

7. Control de Versiones

El desarrollo del proyecto fue gestionado mediante Git y GitHub, aplicando un flujo basado en ramas de funcionalidad y Pull Requests. Esta estrategia permite documentar el avance por etapas y mantener un historial ordenado del proceso de construcción del cliente RESTful.

Pull Request	Rama sugerida	Propósito
PR #1	feature/proyecto-base-react	Configurar el proyecto base React con Vite.
PR #2	feature/rutas-cliente-http	Agregar rutas, navegación y configuración de Axios.
PR #3	feature/crud-usuarios	Implementar consumo REST de usuarios.
PR #4	feature/crud-pedidos	Implementar consumo REST de pedidos.
PR #5	feature/interfaz-validaciones	Agregar mejoras visuales, validaciones y mensajes.
PR #6	feature/documentacion-entrega	Agregar documentación en Word y PDF para la entrega.

8. Problemas Encontrados y Solución Aplicada

Durante la integración entre frontend y backend pueden presentarse problemas de conexión, errores de CORS, diferencias entre los nombres de campos esperados por el backend y los enviados desde el frontend, o respuestas de error no controladas. Para reducir estos inconvenientes, se configuró una URL base en .env, se centralizó el consumo HTTP en servicios independientes y se implementó un interceptor de Axios para normalizar los mensajes de error.

Otra dificultad frecuente en este tipo de integración es la validación de formularios. Para resolverla, los modelos de usuarios y pedidos incluyen funciones de validación que revisan campos obligatorios antes de enviar la información al servidor. Esto mejora la experiencia del usuario y evita peticiones incompletas.

9. Conclusiones

El desarrollo de la Actividad 4 permitió completar el ciclo de integración entre una API backend y un cliente frontend, evidenciando el consumo real de endpoints RESTful desde una interfaz gráfica. React facilitó la construcción modular de las pantallas, mientras que Axios permitió centralizar y reutilizar la lógica de comunicación HTTP.

La aplicación implementada cumple con las operaciones CRUD para usuarios y pedidos, integrando métodos GET, POST, PUT y DELETE. Además, la separación del código en componentes, páginas, servicios, modelos y configuración contribuye a una estructura clara, mantenible y coherente con buenas prácticas de desarrollo web.

Finalmente, el uso de Git y GitHub mediante ramas y Pull Requests permitió conservar evidencia del proceso de desarrollo, lo cual favorece la trazabilidad del proyecto y fortalece prácticas profesionales aplicables en contextos académicos y laborales.

Referencias

- Axios. (s. f.). Axios documentation. <https://axios-http.com/docs/intro>
- Chacon, S., & Straub, B. (2014). Pro Git (2.a ed.). Apress. <https://git-scm.com/book/en/v2>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine). <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Meta Open Source. (s. f.). React documentation. <https://react.dev/>
- React Router. (s. f.). React Router documentation. <https://reactrouter.com/>
- Universidad de Cartagena. (2026). Actividad Unidad 4 - Protocolo Colaborativo - Desarrollo de Web.
- Vite Team. (s. f.). Vite documentation. <https://vite.dev/guide/>